

- REMO_DiRel 汇报文档
 - 1. 项目一句话概括
 - 2. 先理解问题：我们到底在解决什么
 - 2.1 什么是昂贵多目标优化
 - 2.2 为什么超多目标更难
 - 3. 从 REMO 到 REMO_DiRel：我们是怎么想到创新的
 - 3.1 原始 REMO 的基本思想
 - 3.2 原始 REMO 的局限
 - 3.3 我们的创新思路是如何形成的
 - 4. 整体架构总览
 - 4.1 文件级架构
 - 4.2 主流程图
 - 5. 代码级主流程详解
 - 5.1 入口类：REMO_DiRel.m
 - 5.1.1 参数设置
 - 5.1.2 初始化
 - 5.1.3 历史状态
 - 5.1.4 每一代的执行顺序
 - 5.1.5 容错设计
 - 6. 模块一：难度感知目标画像
 - 6.1 我们为什么需要这个模块
 - 6.2 代码中怎样定义目标难度
 - 6.3 为什么还需要 RefineEasySubset
 - 6.4 这一创新为什么有效
 - 7. 模块二：双尺度关系学习网络
 - 7.1 为什么仍然坚持“关系学习”
 - 7.2 关系对是怎样构造的
 - 7.3 双尺度网络具体怎么训练
 - 7.4 “迁移初始化”到底做了什么
 - 7.5 为什么不是只用于模型
 - 8. 模块三：逐候选逆方差仲裁
 - 8.1 为什么需要仲裁器
 - 8.2 代码中怎样得到均值和方差
 - 8.3 逐候选逆方差加权
 - 8.4 冲突分支设计
 - 8.5 代理筛选怎样接入 GA
 - 9. 三大创新如何串成一个完整方法

- 10. 为什么我们的创新是有效的
- 10.1 理论上为什么合理
 - 10.1.1 关系学习比目标回归更适合小样本
 - 10.1.2 易目标子空间能降低关系边界复杂度
 - 10.1.3 全目标与子目标是互补而不是替代
 - 10.1.4 逐候选仲裁比固定融合更符合代理模型本质
- 10.2 代码中怎样为有效性预留验证
- 10.3 目前实现中还能拿来诊断的量
- 11. 和其他方法相比，我们的定位
- 12. 论文应该怎样写
- 12.1 摘要怎么写
- 12.2 引言应该怎么铺
- 12.3 相关工作怎么分
- 12.4 方法章节怎么组织
 - 3.1 Overall Framework
 - 3.2 Difficulty-Aware Objective Profiling
 - 3.3 Dual-Scale Relation Learning
 - 3.4 Candidate-wise Arbitration
 - 3.5 Surrogate-Assisted Search
- 12.5 实验章节怎么写
- 12.6 结论怎么写
- 13. 汇报时可以直接使用的讲解顺序
- 14. 当前代码实现与论文表述时要注意的地方
 - 14.1 默认主路径不是 Kriging 难度
 - 14.2 “共享 backbone” 在代码中的准确表达
 - 14.3 消融实验要和当前主版本参数对齐
 - 14.4 文本编码问题
- 15. 最后总结

REMO_DiRel 汇报文档

1. 项目一句话概括

REMO_DiRel 是一个面向昂贵多/超多目标优化问题的关系学习代理算法。它不直接拟合每个目标函数的数值，而是学习“两个解谁更值得保留”的相对关系；在此基础上，它进一步提出：

1. 在线识别各目标当前阶段的难易程度；
2. 同时训练“全目标关系模型”和“易目标子集关系模型”；
3. 对每一个候选解，根据两个模型各自的不确定性进行动态仲裁；
4. 用更少的真实函数评估，尽量找到更好的 Pareto 前沿近似。

如果把原始 REMO 理解为“用一个老师给所有候选解打分”，那么 REMO_DiRel 的核心思想就是：

当目标很多、目标之间难度不一致时，不应该只听一个老师，而应该让“全局老师”和“局部更可靠的老师”共同判断，并且谁更自信就多听谁。

2. 先理解问题：我们到底在解决什么

2.1 什么是昂贵多目标优化

多目标优化中，一个解同时对应多个目标值，例如：

- 成本要低；
- 能耗要低；
- 性能要高；
- 可靠性也要高。

这些目标通常互相冲突，不存在一个解在所有目标上都最好，因此我们寻找的是一组折中解，即 Pareto 解集。

“昂贵”指的是一次真实评估代价很高，例如：

- 一次有限元仿真要几分钟甚至几小时；
- 一次实验要消耗材料；
- 一个复杂模型评估非常慢。

所以算法不能像普通进化算法那样随便评估几万次，而是要在很小的真实评估预算下，尽可能聪明地挑出值得评估的新解。

2.2 为什么超多目标更难

当目标数从 2、3 个增加到 5、10、20 个时，会出现几个典型问题：

1. 支配关系退化

很多解之间互不支配，传统 Pareto 比较越来越难区分好坏。

2. 不同目标难度并不一致

有些目标在当前种群附近很容易建模，有些目标变化复杂、停滞明显、与其他目标冲突强。

3. 把所有目标一视同仁，会让代理模型被“最难的部分”拖累

如果直接在全目标空间里训练一个统一模型，模型要同时解释所有目标形成的复杂关系，数据量又少，容易不稳定。

4. 同一个模型并不是在所有候选区域都可靠

一个模型对某些候选点预测很稳，对另一些点却可能分歧很大。只用全局固定权重融合模型，无法反映这种局部差异。

3. 从 REMO 到 REMO_DiRel：我们是怎么想到创新的

3.1 原始 REMO 的基本思想

原始 REMO 的主流程在 `REMO/REMO.m` 中：

1. 用拉丁超立方初始化种群；
2. 通过 `RefSelect.m` 选出一组参考解；
3. 用 `GetOutput_PBI.m` 按 PBI 规则把当前解分成“好解”和“非好解”；
4. 用 `GetRelationPairs.m` 构造解对，标签为：
 - 1：前一个解优于后一个解；
 - 0：二者同类；
 - -1：前一个解劣于后一个解；
5. 训练一个 `patternnet` 关系分类器；
6. 用关系模型辅助遗传搜索，在大量候选解中筛出少量真正送去昂贵评估的解。

这个设计很聪明，因为它避免了直接预测高维目标值，而是只学习“谁比谁更好”的关系。对于昂贵优化，小样本下关系学习往往比数值回归更稳。

3.2 原始 REMO 的局限

但是当目标数很多时，一个单一全目标关系网络会遇到三个问题：

1. 所有目标被混在一起处理

容易目标和困难目标共享一个标签空间，难目标会让关系边界更复杂。

2. 模型只有一个视角

全目标模型擅长维护整体 Pareto 方向，但不一定能敏锐捕捉局部更容易学习的结构。

3. 候选点之间的置信度差异没有被充分利用

对某个候选点，模型可能非常确定；对另一个候选点，同一个模型却很犹豫。原始单模型无法表达“这个点上我更信谁”。

3.3 我们的创新思路是如何形成的

我们的设计逻辑可以概括为一条很自然的推理链：

1. 既然不同目标难度不同，就先识别难度。
2. 既然某些目标更容易学，就单独建一个“易目标子空间”模型。
3. 既然全目标模型保留全局信息、子空间模型在局部更可靠，就两个都保留。
4. 既然两个模型在不同候选点上的可靠性不同，就不能固定平均，而要逐候选解仲裁。
5. 既然子空间标签和全目标标签有相关结构，就让子模型从全模型迁移初始化，而不是完全从零开始。

因此，REMO_DiRel 的三项主创新并不是彼此孤立的技巧，而是围绕同一个问题逐层展开：

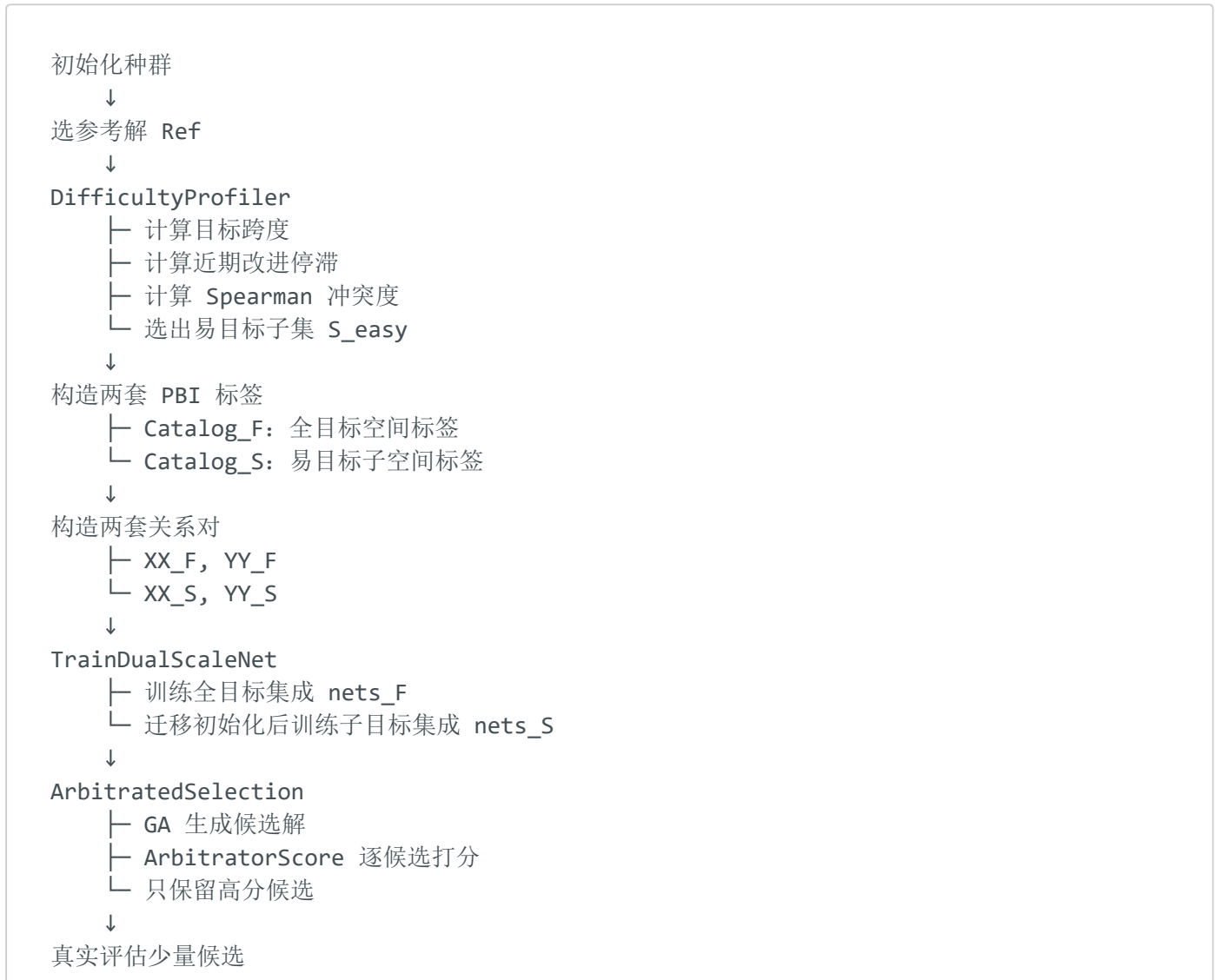
- 目标很多，且难度不均；
- 所以要做难度感知；
- 有了难度感知，就可以做双尺度关系学习；
- 有了双尺度模型，就需要一个能体现局部可靠性的仲裁器。

4. 整体架构总览

4.1 文件级架构

文件	作用
REMO_DiRel.m	总控入口，负责初始化、主循环、调用各模块
DifficultyProfiler.m	在线评估目标难度并选择易目标子集
ConflictDegree.m	计算目标间 Spearman 冲突度
RefineEasySubset.m	去除易目标子集内部冗余
GetRelationPairsBudgeted.m	构造带预算上限的平衡关系对
TrainDualScaleNet.m	训练全目标与子目标双关系网络
TransferFineTune.m	将全目标网络权重迁移初始化到子目标网络
ArbitratorScore.m	计算双模型逐候选逆方差融合分数
ArbitratedSelection.m	在 GA 生成的候选解中做代理筛选
KrigingNRMSE.m	旧版/诊断型难度指标，目前不进入默认主路径

4.2 主流程图



5. 代码级主流程详解

5.1 入口类：REMO_DiRel.m

主函数 `main(Algorithm, Problem)` 是整个算法的调度中心。

5.1.1 参数设置

```
[k_easy_user,tau_conf,alpha,k,gmax,K_ens,win_K] = ...
    Algorithm.ParameterSet(-1, 0.3, 0.6, 6, 1000, 3, 3);
```

含义如下：

参数	含义
k_easy_user	易目标子集大小，-1 时自动取约一半目标
tau_conf	仲裁时判定高/低不确定性的阈值
alpha	难度公式中“建模难度”和“冲突难度”的平衡系数
k	参考解个数
gmax	每代代理模型最多筛选多少候选
K_ens	bagging 集成规模
win_K	难度滑动窗口长度

5.1.2 初始化

```
if Problem.D <= 10
    N = 11*Problem.D - 1;
else
    N = 100;
```

```
end
PopDec      = UniformPoint(N,Problem.D,'Latin');
Population  = Problem.Evaluation(...);
Archive     = Population;
```

这里延续了 REMO 的初始化逻辑：

- 决策维度较小时，初始样本数为 $11D - 1$ ；
- 决策维度较大时，固定为 100 ；
- 使用拉丁超立方采样保证初始点覆盖更均匀。

5.1.3 历史状态

```
H.d_score = nan(Problem.M, win_K);
H.model    = nan(Problem.M, win_K);
H.improve  = nan(Problem.M, win_K);
H.conf     = nan(Problem.M, win_K);
H.best     = nan(Problem.M, 1);
```

H 是一个跨代状态缓存，用于记录：

- 每个目标的难度得分；
- 建模难度分量；
- 改进停滞分量；
- 冲突度分量；
- 历史最好值。

这说明算法不是“每代重新猜一次哪些目标容易”，而是对难度做了短期时序平滑。

5.1.4 每一代的执行顺序

主循环中的核心逻辑如下：

```
Ref = RefSelect(Population, k);
[d_score, H, S_easy] = DifficultyProfiler(Population, H, gen, alpha, k_easy);
```

第一步，先用 `RefSelect` 选当前代表性参考解；再用 `DifficultyProfiler` 得到：

- `d_score`：每个目标的当前难度；

- 更新后的历史 **H**;
- **S_{easy}**: 本代选出的易目标集合。

接着构造全目标标签:

```
Catalog_F = GetOutput_PBI(PopObj, RefObj);
[XX_F,YY_F] = GetRelationPairsBudgeted(Input, Catalog_F, pairMax);
```

然后构造子目标标签:

```
PopObjSub = PopObj(:, S_easy);
Ref_S_obj = UniformPoint(size(RefObj,1), M_sub, 'ILD');
P_min = min(PopObjSub, [], 1);
P_span = max(max(PopObjSub, [], 1) - P_min, 1e-12);
Ref_S_obj = Ref_S_obj .* P_span + P_min;
Catalog_S = GetOutput_PBI(PopObjSub, Ref_S_obj);
[XX_S,YY_S] = GetRelationPairsBudgeted(Input, Catalog_S, pairMax);
```

这里有一个很重要的工程细节:

子目标空间里的参考向量并不是直接沿用全目标参考解，而是重新在子空间中生成，并缩放到当前子目标取值范围。

否则 **Catalog_S** 可能退化，导致子模型学不到有效关系。

之后训练双网络:

```
DualNet = TrainDualScaleNet(XX_F, YY_F, XX_S, YY_S, K_ens);
```

再交给仲裁选择:

```
Next = ArbitratedSelection(Problem, Ref, Input, gmax, Smodel);
```

最后是真实评估与存档更新:

```
Next = sanitizeCandidates(Next, Problem, Archive, remain);
Archive = [Archive, Problem.Evaluation(Next)];
Population = RefSelect(Archive, Problem.N);
```

5.1.5 容错设计

主文件中还有三个实用的小函数：

- `fallbackOffspring`：当双模型不可用时，退回普通 GA 产生后代；
- `sanitizeCandidates`：做边界裁剪、去重、去除已评估样本、按剩余预算截断；
- `randomFill`：如果清洗后没有候选，则补入少量随机点。

这说明项目不是只追求理论表达，也考虑了在真实运行中避免某一代因样本退化而完全卡死。

6. 模块一：难度感知目标画像

对应文件：

- `DifficultyProfiler.m`
- `ConflictDegree.m`
- `RefineEasySubset.m`

6.1 我们为什么需要这个模块

在超多目标场景中，不同目标的局部行为往往不一样：

- 有些目标当前变化范围大，难以稳定拟合；
- 有些目标已经几代没有明显改进；
- 有些目标与很多其他目标强冲突；
- 也有些目标和其他目标高度冗余。

如果不区分这些差异，而是每次都随机选子目标，或者固定按编号分组，那么“子目标模型”就未必真的更容易学习。

所以第一项创新不是“随便取一部分目标”，而是：
在线识别当前阶段最适合作为易学习视角的目标子集。

6.2 代码中怎样定义目标难度

`DifficultyProfiler.m` 中，模型难度由两部分组成：

```
spanScore = minmaxNorm(log1p(max(spanRaw, 0)));  
improveScore = 1 - minmaxNorm(relImprove);  
modelDifficulty = 0.5 .* spanScore + 0.5 .* improveScore;
```

含义是：

1. 目标跨度越大，建模越难

当前种群在该目标上的取值范围越大，关系结构可能越复杂。

2. 近期改进越停滞，难度越高

如果该目标的最好值已经很久没有改善，说明当前搜索在这个目标上推进困难。

然后 `ConflictDegree.m` 计算每个目标与其余目标的平均 Spearman 冲突程度：

```
conf(j) = mean(1 - rho_abs(j, others));
```

这里使用 $1 - |\rho|$ ，不是简单地看正负相关，原因是：

- $|\rho|$ 很高说明信息高度重复；
- $1 - |\rho|$ 高说明该目标与其他目标关系更独立、更不一致。

在 `DifficultyProfiler.m` 中，最终难度写为：

```
confN = minmaxNorm(confRaw);  
conflictDifficulty = 1 - confN;  
d_now = alpha .* modelDifficulty + (1-alpha) .* conflictDifficulty;
```

也就是说：

- `modelDifficulty` 描述“这个目标当前好不好学”；
- `conflictDifficulty` 描述“这个目标在多目标关系里是否适合作为易子空间的一员”；
- `alpha` 控制二者权衡。

然后用 `win_K` 代滑动平均抑制代际波动：

```
H.d_score(:, col) = d_now;
```

```
d_score = meanNoNaN(H.d_score, 2);
```

6.3 为什么还需要 RefineEasySubset

只按难度排序会有一个问题：

最容易的几个目标可能彼此几乎重复，例如 Spearman 相关系数接近 1。

如果子集里全是重复目标，那么所谓“易目标子空间”其实只是把同一类信息重复了几遍，不能提供足够多样的局部结构。

所以 RefineEasySubset.m 会检测：

```
if abs(rho(S_easy(i), S_easy(j))) > 0.95
```

当两个候选易目标高度冗余时：

- 保留难度更小的那个；
- 丢掉难度更大的那个；
- 再从剩余目标中按难度顺序补位。

这一步让 S_easy 既“容易学”，又尽量“不重复”。

6.4 这一创新为什么有效

它有效的原因不是“易目标一定比难目标更重要”，而是：

1. 在小样本代理建模里，先学稳的结构更有价值；
2. 易目标子集上的 PBI 标签通常更清晰，关系边界更简单；
3. 子模型提供的是一个“低噪声辅助视角”，不是取代全目标；
4. 动态选择比固定分组更能适应不同问题、不同搜索阶段。

论文中可以把这一点表述为：

我们不是人为假设某几个目标永远简单，而是根据当前种群状态在线估计目标难度，使子模型始终围绕“当前最可学习”的目标结构展开。

7. 模块二：双尺度关系学习网络

对应文件：

- `GetRelationPairsBudgeted.m`
- `TrainDualScaleNet.m`
- `TransferFineTune.m`

7.1 为什么仍然坚持“关系学习”

我们没有改成直接回归目标值，原因有三点：

1. 真实样本少，回归每个目标的精确数值很难；
2. 优化最终关心的是“谁更值得选”，而不是每个目标的绝对值；
3. REMO 已经证明关系学习适合昂贵优化，我们是在此基础上增强，而不是推翻。

7.2 关系对是怎样构造的

原始 REMO 的 `GetRelationPairs.m` 会枚举大量解对。

`REMO_DiRel` 中改为 `GetRelationPairsBudgeted.m`，原因很现实：

- 现在我们要训练两套模型；
- 如果还枚举所有关系对，计算量会迅速膨胀。

因此新函数设置：

```
pairMax = 6000;  
perClass = floor(pairMax/3);
```

并尽量平衡三类标签：

- `1`：好解优于坏解；
- `0`：同类关系；
- `-1`：坏解劣于好解。

这项改动的意义有两个：

1. 控制训练成本，保证算法在多目标场景下还能跑得动；
2. 避免某一类关系样本过多，让分类器偏向多数类。

7.3 双尺度网络具体怎么训练

`TrainDualScaleNet.m` 先训练全目标模型：

```
nets_F = trainBagEnsemble(..., K_ens, [], 60);
```

再训练子目标模型：

```
nets_S = trainBagEnsemble(..., K_ens, nets_F, 30);
```

其中：

- `nets_F` 学的是全目标标签 `YY_F`；
- `nets_S` 学的是易目标子空间标签 `YY_S`；
- 每个分支不是单网，而是一个 bagging 集成；
- 全目标训练约 60 epochs；
- 子目标训练约 30 epochs；
- 子目标网络在训练前会用 `nets_F` 对应成员的权重做迁移初始化。

7.4 “迁移初始化” 到底做了什么

`TransferFineTune.m` 里没有冻结层，而是将源网络中形状兼容的：

- 输入权重 `IW`；
- 层间权重 `LW`；
- 偏置 `b`；

复制到目标网络中。

也就是说：

```
net = TransferFineTune(net, initFromNets{i});
```

让 `net_S` 不是从随机起点出发，而是从已经学到全目标关系结构的参数附近出发。

这一点非常重要，因为全目标关系和易目标关系并不是毫无关系：

- 它们共享同一批决策变量；
- 都在描述“好解/坏解”之间的排序结构；
- 只是子模型面对的是更简单、更局部的投影视角。

因此迁移初始化能减少子模型的搜索难度，加快收敛，并在小样本场景下提高稳定性。

7.5 为什么不是只用于子模型

因为子模型只看 `S_easy`，它确实更容易学，但也会丢失困难目标的信息。如果只听子模型，可能过分偏向容易优化的维度，损害整体 Pareto 质量。

所以我们的策略不是“全目标太难，干脆不要”，而是：

- `net_F` 保持全局一致性；
- `net_S` 提供更清晰的局部辅助；
- 后续仲裁器负责判断当前候选点上更该信谁。

8. 模块三：逐候选逆方差仲裁

对应文件：

- `ArbitratorScore.m`
- `ArbitratedSelection.m`

8.1 为什么需要仲裁器

有了两个模型后，马上会遇到一个问题：

两个模型意见不一致时怎么办？

如果简单平均：

- 会默认两个模型永远一样可靠；
- 但实际并不是这样。

如果使用一个全局权重：

- 会默认某个模型在整代所有候选上都更可靠；
- 但同一个模型在不同候选区域上的不确定性可能差很多。

所以我们采用了更细粒度的策略：

对每一个候选解单独算两个模型的方差，谁更稳，谁权重大。

8.2 代码中怎样得到均值和方差

在 `ArbitratorScore.m` 中：

```
[mu_F, sigma2_F] = scoreAllByEnsemble(... nets_F ...);  
[mu_S, sigma2_S] = scoreAllByEnsemble(... nets_S ...);
```

对每个候选点：

- 每个 bagging 成员都会给出一个关系分数；
- 多个成员的平均值是 `mu`；
- 多个成员之间的方差是 `sigma2`。

这就把“模型不确定性”自然转化成了集成成员之间的分歧。

8.3 逐候选逆方差加权

核心公式如下：

```
invF = 1 ./ (s_F.^2 + eps_v);  
invS = 1 ./ (s_S.^2 + eps_v);  
w_F = invF ./ (invF + invS);  
w_S = 1 - w_F;  
base = w_F .* tildeS_F + w_S .* tildeS_S;
```

解释为：

- 方差越小，说明集成成员意见越一致；
- 方差越小，逆方差越大；
- 权重就越高；

- 并且 w_F 、 w_S 是每个候选解自己算自己的，不是全局常数。

这就是项目名中 DiRel 的另一层含义：

不仅目标难度是动态的，两个关系模型的信任程度也是动态的。

8.4 冲突分支设计

如果两个模型符号相反：

```
conflict = (sign(mu_F) .* sign(mu_S)) < 0;
```

则进入特殊处理：

1. 双方都不确定

```
abstain = conflict & (n_F > tau) & (n_S > tau);  
base(abstain) = 0;
```

直接弃权，不让高风险点因为偶然平均而被选中。

2. 子模型支持、全模型反对，且子模型更确定

```
subwin = conflict & (mu_S > 0) & (mu_F < 0) & (n_F > tau) & (n_S <= tau);
```

这类点可能代表“全局视角还没看懂，但易目标子空间发现了潜在新方向”。因此加入基于候选间距离的 novelty 奖励，鼓励探索稀缺区域。

8.5 代理筛选怎样接入 GA

ArbitratedSelection.m 的流程是：

1. 先用 OperatorGA 产生候选；
2. 用 ArbitratorScore 打分；
3. 每轮保留最优的一小批继续繁殖；
4. 直到代理评估量达到 w_{max} ；

5. 最后只把高分点或前 4 个点送去真实评估。

此外，difficultyAwareGAParam 会根据易目标平均难度调整 GA 参数：

```
disC = round(10 + 20*(1-diff));
disM = round(5 + 20*(1-diff));
proM = 1 + 0.5*diff;
```

代码注释中说明：

- 易且仍在改善时，倾向更局部地开发；
- 难且停滞时，倾向更开放地探索。

这让“难度感知”不只影响建模，也会传递到搜索行为。

9. 三大创新如何串成一个完整方法

创新	解决的问题	代码位置	为什么必要
难度感知目标画像	目标太多且难度不均，不能随便选子目标	DifficultyProfiler.m	让子模型始终围绕当前更可学习的目标结构
双尺度关系学习	单一全目标模型负担过重	TrainDualScaleNet.m	同时保留全局一致性与局部易学结构
逐候选逆方差仲裁	两模型可靠性因候选而异	ArbitratorScore.m	让每个候选点使用最适合自己的模型权重

三者是递进关系：

1. 没有难度画像，就不知道怎样构造有意义的子模型；
2. 没有双尺度模型，就谈不上互补信息；
3. 没有逐候选仲裁，双模型只能机械平均，创新价值会被削弱。

10. 为什么我们的创新是有效的

这里要区分“理论直觉”和“实验验证”两层。

10.1 理论上为什么合理

10.1.1 关系学习比目标回归更适合小样本

昂贵优化最稀缺的是样本。

比起拟合绝对目标值，学习相对关系更贴近优化决策本身，也降低了建模难度。

10.1.2 易目标子空间能降低关系边界复杂度

如果某些目标当前跨度较小、仍有改善、且不高度冗余，那么这些目标形成的子空间通常更容易形成稳定关系标签。

子模型因此能提供一个低噪声视角。

10.1.3 全目标与子目标是互补而不是替代

- 全目标：保证不偏离原始多目标任务；
- 子目标：在局部更容易学、能更快给出清晰信号。

这是一种 bias-variance tradeoff:

- 全目标模型偏差更小，但方差可能更大；
- 子目标模型方差更小，但存在信息缺失；
- 融合后可利用二者互补。

10.1.4 逐候选仲裁比固定融合更符合代理模型本质

代理模型的不确定性是位置相关的。

某个模型可能在区域 A 很可靠，在区域 B 很不可靠。

所以“每个候选点单独算权重”比“一整代共享一个权重”更精细。

10.2 代码中怎样为有效性预留验证

仓库中已经准备了五个消融变体：

变体	去掉了什么	验证的问题
REMO_DiRel_noDi	去掉难度排序，改为随机子目标	难度画像是否必要
REMO_DiRel_noSub	去掉子目标模型	双尺度建模是否必要
REMO_DiRel_noTrans	去掉迁移初始化	权重迁移是否有效
REMO_DiRel_AvgArb	改成固定 0.5/0.5 平均	逆方差权重是否优于简单平均
REMO_DiRel_FullArb	改成全局标量权重	逐候选权重是否优于全局权重

这组消融设计非常好，因为它不是泛泛地证明“我们的方法好”，而是逐个拆掉核心机制，回答：

- 1. 是不是“随便一个子模型”都行？
- 2. 是不是“两个模型”本身就足够了？
- 3. 是不是“迁移”真的带来收益？
- 4. 是不是“局部仲裁”确实比平均和全局权重更好？

10.3 目前实现中还能拿来做诊断的量

TrainDualScaleNet.m 保存了：

```
DualNet.p_err_F
DualNet.p_err_S
```

这两个字段可以用于画图或做案例分析：

- 当 $p_err_S < p_err_F$ 时，可以支持“易子空间更容易学习”的论断；
- 当某些代 p_err_S 升高时，可以分析是否因为 S_easy 质量变差；
- 配合 $H.d_score$ ，可以展示目标难度与子模型质量之间的关系。

这部分非常适合写进论文中的机制分析或附录实验。

11. 和其他方法相比，我们的定位

方法	核心做法	局限
REMO	单一全目标关系网络	不能利用目标难度差异
Subproblem_REMO	固定分组、多个分类器	子问题划分静态，不随阶段变化
REMO_SRMaO	用集成不确定性做全局权重	权重更偏全局，不够候选级细致
REMO_new2_AdaMaO	多指标、自适应规则较多	机制较重，参数与规则更多
REMO_DiRel	动态难度排序 + 双尺度网络 + 逐候选仲裁	兼顾适应性、表达力与机制可解释性

我们的卖点不是“结构最复杂”，而是：

- 每一个新增模块都有明确问题对应；
- 每一项机制都有消融可证；
- 整体设计依然沿用 REMO 的关系学习主线，没有丢掉原方法最有价值的部分。

12. 论文应该怎样写

下面给出一套适合投稿论文或毕业论文的方法章节结构。

12.1 摘要怎么写

摘要建议按四句组织：

1. 问题背景

昂贵超多目标优化中，有限预算和高维目标空间使代理建模困难。

2. 现有不足

现有关系学习方法通常在全目标空间使用单一模型，忽略了目标难度随搜索阶段变化及模型可靠性的局部差异。

3. 提出方法

提出 **REMO_DiRel**，包括在线目标难度画像、全目标/易目标双尺度关系学习，以及逐候选逆方差仲裁。

4. 实验结果

在多组 benchmark 与多目标维度上，相比若干代表性算法取得更优或更稳健表现；消融实验验证三个核心模块贡献。

12.2 引言应该怎么铺

引言建议按这个逻辑展开：

1. 昂贵 MOO 很重要；
2. 代理模型是主流思路；
3. 高目标维度下，直接回归和单一代理都更困难；
4. 关系学习减少了数值拟合难度，但现有方法仍默认所有目标同质；
5. 实际上目标难度、冲突关系、局部可靠性都在变化；
6. 因此提出 **REMO_DiRel**。

然后列贡献：

1. 提出一种轻量级在线目标难度画像机制；
2. 提出双尺度关系学习框架，并通过迁移初始化复用全目标关系知识；
3. 提出逐候选逆方差仲裁策略，实现局部自适应融合；
4. 设计系统消融实验验证各模块有效性。

12.3 相关工作怎么分

建议分为三小节：

1. 昂贵多目标/超多目标优化；
2. 关系学习型代理优化；
3. 不确定性融合、迁移学习或目标分解相关方法。

写法上要把自己的位置放清楚：

- 不是普通 surrogate-assisted EMO；
- 不是单纯多任务学习；

- 不是静态分组;
- 而是**面向目标难度异质性的动态双尺度关系学习**。

12.4 方法章节怎么组织

建议按以下顺序:

3.1 Overall Framework

先给总图和算法伪代码, 说明每代发生什么。

3.2 Difficulty-Aware Objective Profiling

写:

- `spanScore`
- `improveScore`
- `ConflictDegree`
- 滑动平均
- 冗余剔除

这里要强调“轻量级”, 因为默认主路径已经不用 Kriging CV, 避免把方法写成高开销版本。

3.3 Dual-Scale Relation Learning

写:

- 全目标标签;
- 易目标标签;
- 预算化关系对构造;
- bagging 集成;
- 迁移初始化。

3.4 Candidate-wise Arbitration

写:

- 集成均值和方差;
- 逆方差权重;
- 冲突弃权;
- 子模型置信优势下的 novelty 奖励。

3.5 Surrogate-Assisted Search

写:

- OperatorGA
- 代理筛选循环;
- 高分阈值;
- 难度感知 GA 参数。

12.5 实验章节怎么写

建议结构:

1. 实验设置

- DTLZ、WFG、MaF;
- $M \in \{5, 10, 15, 20\}$;
- IGD+、HV、运行时间;
- 30 次独立运行;
- Wilcoxon 检验。

2. 与代表算法比较

- 总表;
- 不同目标维度分表;
- 收敛曲线或雷达图。

3. 消融实验

- noDi
- noSub
- noTrans

- AvgArb
- FullArb

4. 机制分析

- S_{easy} 随代变化;
- d_{score} 热力图;
- p_{err}_F 与 p_{err}_S 曲线;
- 候选级权重分布;
- 个别问题上的案例分析。

12.6 结论怎么写

结论不要只说“结果更好”，而要回扣设计逻辑：

1. 目标难度异质性是真实存在且可利用的；
2. 易目标子空间能为昂贵优化提供更稳定的辅助关系信号；
3. 模型融合应当是局部自适应的，而不是固定或全局统一的；
4. 后续可继续探索更细粒度的目标动态分组、显式任务共享、或更强的不确定性估计。

13. 汇报时可以直接使用的讲解顺序

如果是答辩或组会，建议按下面 8 页讲：

1. **问题页**：昂贵超多目标优化为什么难；
2. **基线页**：REMO 怎么做，单关系模型的局限；
3. **观察页**：不同目标难度不同，模型置信度也随候选点变化；
4. **总框架页**：给出 $REMO_DiRel$ 流程图；
5. **创新一**：难度感知目标画像；
6. **创新二**：双尺度关系学习 + 迁移初始化；
7. **创新三**：逐候选逆方差仲裁；
8. **实验页**：总结果 + 消融结果 + 机制可视化。

讲的时候最关键的一句话是：

我们不是简单地多加一个模型，而是先回答“哪些目标当前更值得学”，再回答“怎样让全局与局部信息协同”，最后回答“对每一个候选点究竟该信谁更多”。

14. 当前代码实现与论文表述时要注意的地方

14.1 默认主路径不是 Kriging 难度

`README_REMO_DiRel.md` 和 `DifficultyProfiler.m` 都说明：
默认实现已经从早期的 Kriging 交叉验证难度，改成了更轻量的：

- 目标跨度；
- 改进停滞；
- Spearman 冲突度。

因此论文正文应以当前轻量版本为准。

`KrigingNRMSE.m` 更适合作为：

- 早期版本；
- 诊断指标；
- 或附录中的替代方案。

14.2 “共享 backbone” 在代码中的准确表达

当前代码并不是两个分支物理共享同一个实时 backbone，而是：

- `net_F` 先训练；
- `net_S` 通过复制 `net_F` 权重进行迁移初始化；
- 然后独立微调。

所以论文中更准确的说法是：

dual-scale relation learning with transfer initialization

而不是严格意义上的：

hard parameter sharing backbone

除非后续代码真的实现参数共享。

14.3 消融实验要和当前主版本参数对齐

部分消融目录中的默认参数仍沿用较旧配置，例如：

- `gmax = 3000`
- `K_ens = 5`

而主版本当前默认是：

- `gmax = 1000`
- `K_ens = 3`

正式实验时应统一参数，避免把“参数不同”误当成“机制差异”。

14.4 文本编码问题

部分 `.m` 文件中的中文注释存在乱码显示，但不影响算法逻辑。

正式论文和汇报材料不要直接复制乱码注释，应以代码行为和本报告中的中文解释为准。

15. 最后总结

`REMO_DiRel` 的核心价值，不在于堆叠很多复杂模块，而在于它对一个真实问题给出了完整而连贯的回答：

1. **超多目标并不均质**，所以先判断哪些目标当前更容易学；
2. **全目标关系和易目标关系各有长处**，所以同时建模；
3. **不同候选点上的可靠性不同**，所以逐候选动态仲裁；
4. **小样本下训练成本很贵**，所以使用关系学习、预算化关系对、bagging 和迁移初始化共同控制风险。

如果把这套方法写成论文，它最有说服力的地方是：

每一项创新都能在代码中找到具体落点，每一项创新都能通过对应消融单独验证，每一项创新都服务于同一个中心问题：如何在昂贵、超多目标、样本稀缺的条件下，做出更可靠的新解选择。